

Asteroid Miner

A Hybrid Architecture for Route Optimization in a 2D Mining Game

Authors: Luis Alejandro Morales | Juan Manuel Otálora | Julián Felipe Donoso

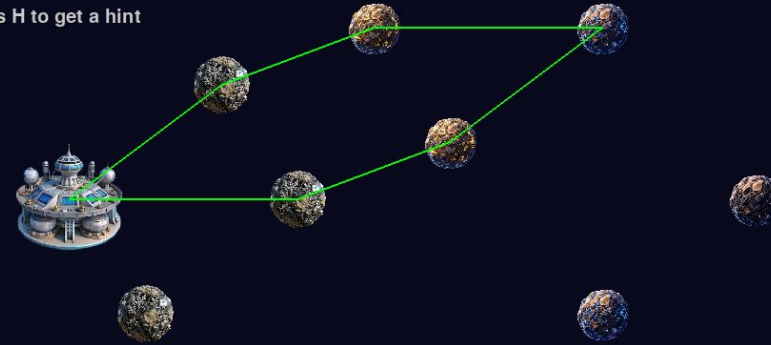
Universidad Distrital Francisco José de Caldas

The Problem

- ✖ Explore an unknown asteroid field.
- 💎 Collect maximum resource value.
- 🛢️ Manage limited and critical fuel.
- 🏠 Guarantee a safe return to base.

Challenge: Finding efficient routes under tight resource constraints.

Fuel: 1.87
Value: 450
Press ENTER to finish
Press H to get a hint



You Win Stars: 3

Press ESC to return

Project Objectives

General Objective

Design an educational video game that integrates algorithms and data structures to solve optimization problems.

Specific Objectives

-  Implement Greedy and Backtracking algorithms.
-  Use Linked List for route history.
-  Implement BST for the asteroid catalog.
-  Integrate Python for UI and C++ for game logic.

General Architecture



Python (Pygame)

Manages the graphical interface, player interaction, and main game loop logic.



JSON

Communication layer for data exchange between the two languages.



C++ (Core)

Processes complex algorithms and manages efficient data structures (BST, Linked List).

Game Levels

Level	Asteroids	Fuel Capacity	Complexity
Level 1	8 Asteroids	18 Units	Basic
Level 2	12 Asteroids	25 Units	Medium
Level 3	16 Asteroids	32 Units	High

Progressive complexity challenges the efficiency of implemented algorithms.

Greedy Algorithm

Heuristic

Score = Value/Distance

Selects the asteroid with the best Value/Distance ratio at each step.

Pros

O(n) speed, providing immediate hints.

Cons

Algorithmic myopia: does not guarantee global optimality.

Backtracking Algorithm



Exhaustive Search

Systematically explores all possible permutations to find the maximum resource collection.

- ✓ Checks fuel at every branch.
- ↶ Performs backtracking on invalid routes.
- 🏆 Updates the best found solution.

Backtracking Pruning

Avoiding Combinatorial Explosion

Without pruning, the algorithm would explore $n!$ routes. Pruning stops branches that cannot surpass the current best solution.

If: $V_{\text{current}} + V_{\text{remaining}} \leq V_{\text{best}} \rightarrow \text{Prune}$



This drastically reduced explored nodes in Level 3, allowing response times under 1 second.

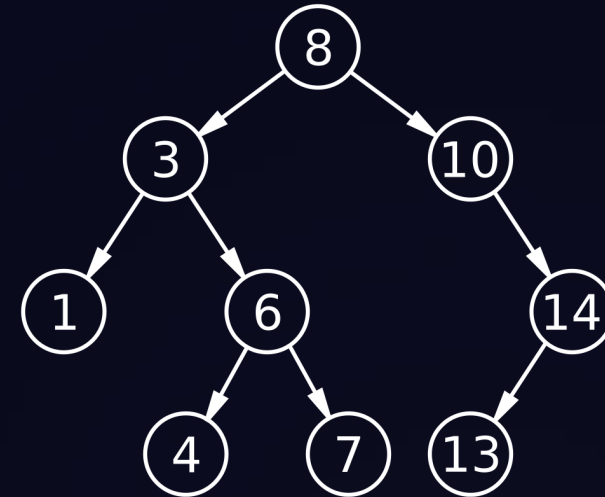
Data Structures



Linked List

Stores **Route History**. Each node represents a mined asteroid in chronological order.

Base -> Ast1 -> Ast5 -> Base

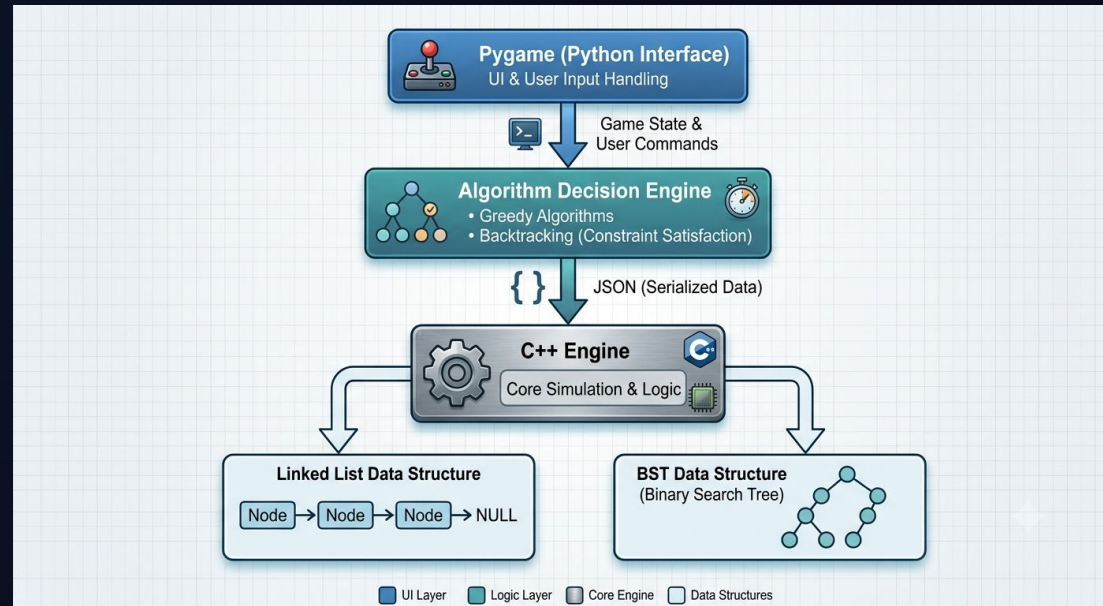


BST (Binary Search Tree)

Sorted catalog of asteroids by **value**. Facilitates resource searching and visualization.

Silver(50) < Gold(100) < Diamond(250)

Python – C++ Communication



Python saves state to JSON.



C++ loads data structures.



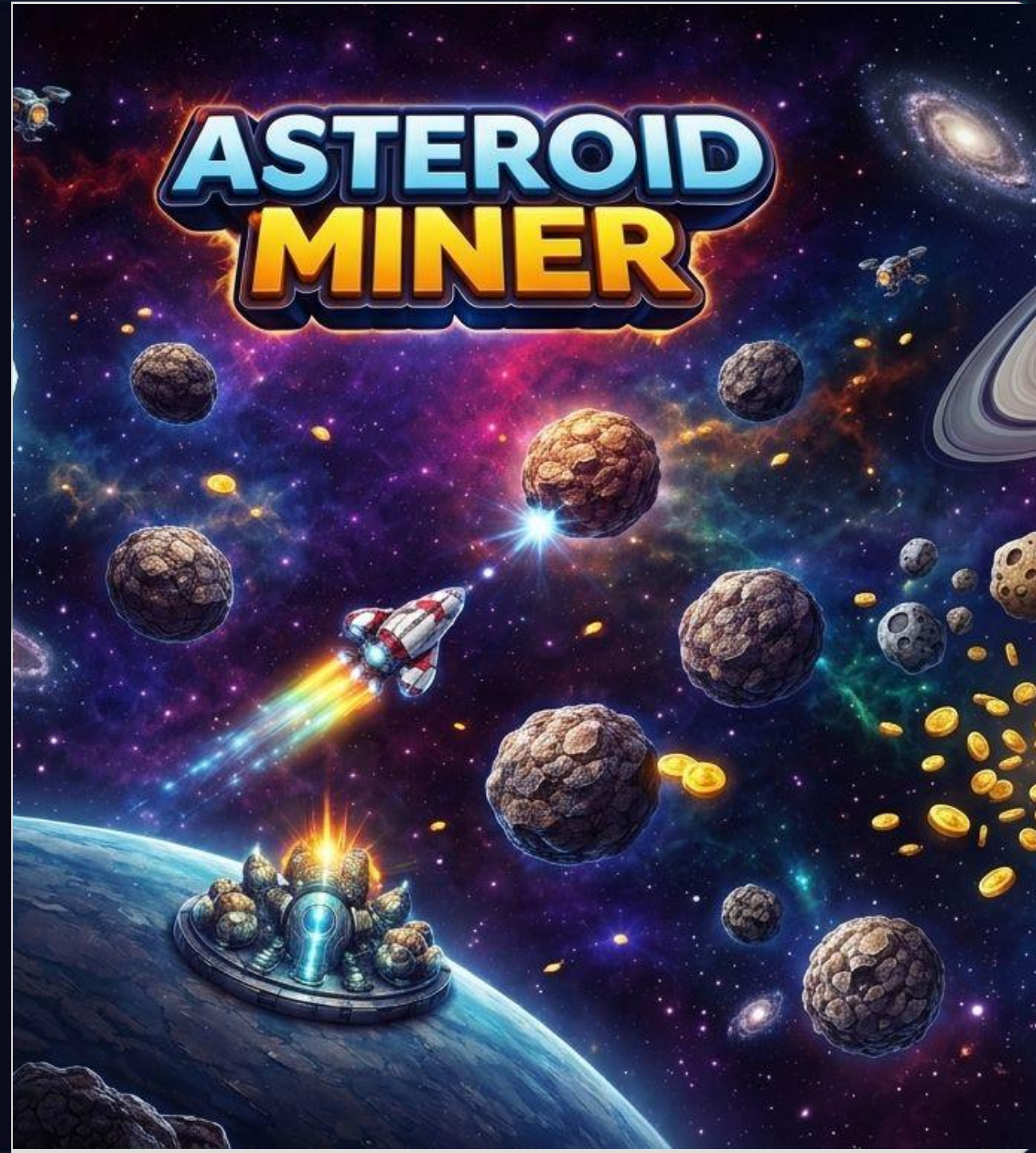
C++ processes algorithms.



Python displays output results.

Demonstration

- ▶ Dynamic level selection.
- 👉 Interactive mining.
- 💡 **Greedy Hint:** Real-time help.
- 📈 Post-mission stats.
- 📁 BST and List visualization.



Conclusions

- ✓ Successful **Python + C++** integration.
- ✓ Functional **Greedy** support.
- ✓ **Backtracking with pruning** ensures optimality.
- ✓ **Linked List** for route history.
- ✓ **BST** for efficient catalog management.
- ✓ Robust data persistence via **JSON**.

Thank you for your attention!